

HDOC Day-28

- 1) Write a C program to:
 - a) Read the number of rows and columns from the user. Ensure that both values are positive. If not, repeat to get the desired input(s).
 - b) Declare a variable-length 2D integer array using the given dimensions.
 - c) Read the array elements column by column using nested loops, with the outer loop iterating over columns and the inner loop over rows.
 - d) Print the elements in row-major order using a single loop and a linear index

- 2) Write a C program that reads elements into a variable-length 2D array, copies them into a 1D array in row-major order, and displays both arrays

Algorithm

1. Start.
2. Read rows and repeat until rows > 0.
3. Read columns and repeat until columns > 0.
4. Declare int a[rows][columns].
5. Using nested loops with columns as the outer loop and rows as the inner loop, read a[i][j].
6. Use one loop k = 0 to rows*columns-1 and print the contiguous row-major elements using *((int *)a + k).
7. Stop.

C Program:

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int rows, cols;
```

```
    // Read positive number of rows
```

```
    do
```

```
    {
```

```
        printf("Enter number of rows: ");
```

```
        scanf("%d", &rows);
```

```

    if (rows <= 0)
        printf("Rows must be positive. Try again.\n");

} while (rows <= 0);

// Read positive number of columns
do
{
    printf("Enter number of columns: ");
    scanf("%d", &cols);

    if (cols <= 0)
        printf("Columns must be positive. Try again.\n");

} while (cols <= 0);

// Variable-length 2D array
int arr[rows][cols];

// Input elements column-wise
printf("\nEnter elements column-wise:\n");

for (int j = 0; j < cols; j++)
{
    for (int i = 0; i < rows; i++)
    {
        printf("arr[%d][%d] = ", i, j);
        scanf("%d", &arr[i][j]);
    }
}

// Display elements in row-major order using one loop
printf("\nElements in row-major order:\n");

for (int k = 0; k < rows * cols; k++)
{
    int i = k / cols;
    int j = k % cols;

    printf("%d ", arr[i][j]);
}

```

```

printf("\n");

return 0;
}

```

Test Case Table:

Case	Rows	Cols	Input order	Expected row-major output
Valid	2	3	1 4 2 5 3 6	1 2 3 4 5 6
Boundary	1	1	9	9
Invalid then valid	0 -> 2	-1 -> 2	1 3 2 4	1 2 3 4

Compilation and Execution:

Test Case 1:

```

(kali㉿kali)-[~/Desktop]
$ ./hdoc28
Enter rows: 2
Enter columns: 3

Enter array elements column by column:
a[0][0] = 1 4 2 5 3 6
a[1][0] = a[0][1] = a[1][1] = a[0][2] = a[1][2] =
Elements in row-major order:
1 2 3
4 5 6

```

Test Case 2:

```

(kali㉿kali)-[~/Desktop]
$ ./hdoc28
Enter rows: 1
Enter columns: 1

Enter array elements column by column:
a[0][0] = 9

Elements in row-major order:
9

```

Test Case 3:

```
(kali㉿kali)-[~/Desktop]
$ ./hdoc28
Enter rows: 0
Rows must be positive.
Enter rows: 2
Enter columns: -1
Columns must be positive.
Enter columns: 2

Enter array elements column by column:
a[0][0] = 1 3 2 4
a[1][0] = a[0][1] = a[1][1] =
Elements in row-major order:
1 2
3 4
```

2) Copy 2D Array into 1D Array in Row-major Order

Algorithm

1. Start.
2. Read the number of rows and columns.
3. Declare a variable-length 2D array.
4. Declare a 1D array of size rows × cols.
5. Read elements into the 2D array.
6. Display the 2D array.
7. Copy each 2D element into the 1D array using:
`oneD[i * cols + j] = arr[i][j]`
8. Display the 1D array.
9. Stop.

C Program

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
    int rows, cols;
```

```
    printf("Enter number of rows: ");
```

```
    scanf("%d", &rows);
```

```
    printf("Enter number of columns: ");
```

```
    scanf("%d", &cols);
```

```
    // Variable-length 2D and 1D arrays
```

```
    int arr[rows][cols];
```

```
    int oneD[rows * cols];
```

```

// Read elements into 2D array
printf("\nEnter elements of the 2D array:\n");

for (int i = 0; i < rows; i++)
{
    for (int j = 0; j < cols; j++)
    {
        printf("arr[%d][%d] = ", i, j);
        scanf("%d", &arr[i][j]);
    }
}

// Display 2D array
printf("\n2D Array:\n");

for (int i = 0; i < rows; i++)
{
    for (int j = 0; j < cols; j++)
    {
        printf("%d ", arr[i][j]);
    }
    printf("\n");
}

// Copy 2D array into 1D array in row-major order
for (int i = 0; i < rows; i++)
{
    for (int j = 0; j < cols; j++)
    {
        oneD[i * cols + j] = arr[i][j];
    }
}

// Display 1D array
printf("\n1D Array in row-major order:\n");

for (int i = 0; i < rows * cols; i++)
{
    printf("%d ", oneD[i]);
}

```

```

printf("\n");

return 0;
}

```

Test Case Table:

Case	Rows	Cols	2D input	Expected 1D array
Valid	2	3	1 2 3 4 5 6	1 2 3 4 5 6
Boundary	1	1	7	7
Valid	3	2	1 2 3 4 5 6	1 2 3 4 5 6

Compilation and Execution:

Test Case 1:

```

(kali@kali)-[~/Desktop]
$ ./hdoc28_q2
Enter rows: 2
Enter columns: 3

Enter elements of the 2D array:
a[0][0] = 1 2 3 4 5 6
a[0][1] = a[0][2] = a[1][0] = a[1][1] = a[1][2] =
2D array:
1 2 3
4 5 6

1D array in row-major order:
1 2 3 4 5 6

```

Test Case 2:

```

(kali@kali)-[~/Desktop]
$ ./hdoc28_q2
Enter rows: 1
Enter columns: 1

Enter elements of the 2D array
a[0][0] = 7

2D array:
7

1D array in row-major order:
7

```

Test Case 3:

```
(kali㉿kali)-[~/Desktop]
$ ./hdoc28_q2
Enter rows: 3
Enter columns: 2

Enter elements of the 2D array:
a[0][0] = 1 2 3 4 5 6
a[0][1] = a[1][0] = a[1][1] = a[2][0] = a[2][1] =
2D array:
1 2
3 4
5 6

1D array in row-major order:
1 2 3 4 5 6
```